

Portfolio

Full Stack Developer | 백종현

전체 흐름을 이해하고 구현하는 개발자

HISTORY

사용자의 경험과
시스템을 함께 고민하는
백종현입니다.



ABOUT.

이름 : 백종현 (1996. 03. 10)

연락처 : 010-8722-9185

이메일 : uoou9677@gmail.com

Portfolio site

: <https://jhyeon9185.github.io/>

EXPERIENCE.

2025.10 ~ 2026.04

[기업연계형] 챗 GPT 활용 및
Devops(데브옵스) 개발자 부트캠프

2015

잠실고등학교 졸업

[자격증]

- 프로그래밍 기능사
- GTQ 포토샵
- GTQ 일러스트

SKILL.

Main Stack



Experienced



기술 전체 그림

백엔드부터 프론트엔드까지 — 전체 흐름을 이해하고 직접 구현합니다

BACKEND ← 62탕냥이 프로젝트

LANGUAGE / FRAMEWORK

- Java 21
- Spring Boot 3.2
- Spring MVC

SECURITY / AUTH

- Spring Security
- JWT
- OAuth2
- BCrypt

DATABASE / ORM

- JPA
- MySQL 8.0
- JPQL

FRONTEND ← DAYPOO 프로젝트

LANGUAGE / FRAMEWORK

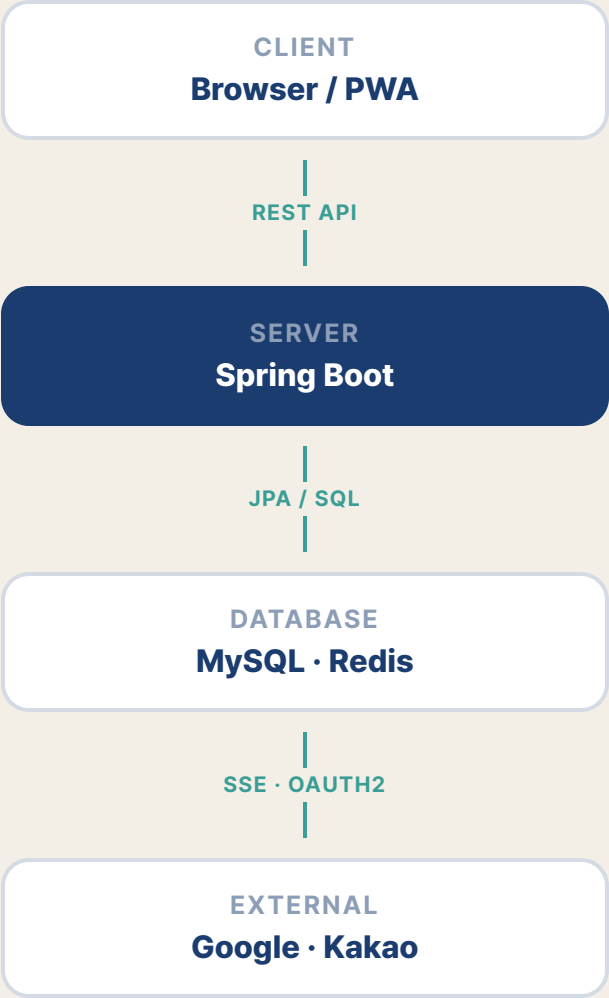
- React
- JavaScript
- TypeScript

BUILD / PERFORMANCE

- Vite
- 번들 66% 감소
- Code Splitting
- Tree Shaking

UI / REAL-TIME / INFRA

- Tailwind CSS
- SSE
- PWA
- Redis



62당냥이

인증 흐름 다이어그램



기간 / 역할

2026.01 ~ 2026.02 (팀 프로젝트)

백엔드 개발 · Spring Security · JWT 설계

기술 스택

Java 21 · Spring Boot 3.2 · Spring Security

JWT · JPA · MySQL 8.0 · Google/Kakao OAuth2

주요 기여

- JWT 발급·검증 구조 직접 설계 (Access + Refresh 이중 토큰)
- Spring Security 필터 체인 구성 및 Stateless 인증 처리
- Google · Kakao OAuth2 소셜 로그인 직접 구현
- 역할 기반 API 접근 제어 (USER / SHELTER_ADMIN / SUPER_ADMIN)
- BCrypt 비밀번호 암호화 및 저장 신뢰성 검증 로직 추가

JWT 인증 설계

/ Stateless 기반 토큰 인증 구조

TOKEN 구조

Access Token

유효 1시간

subject: email

role: USER

userId: 1

HMAC-SHA256

Refresh Token

유효 7일

subject: email

type: 'refresh'

일반 API 접근 차단

요청 흐름

HTTP 요청



JWT Filter



Claims 파싱



SecurityContext



API 처리

JwtAuthenticationFilter 동작

1

Bearer 토큰 추출

Authorization 헤더에서 토큰 분리

OncePerRequestFilter — 요청당 1회만 실행

2

토큰 검증 및 Claims 파싱

서명 검증 후 email · role · userId 추출

3

SecurityContext 등록

인증 객체 생성 후 컨텍스트에 설정

→ DB 조회 없이 인증 완료

역할 기반 접근 제어 (RBAC)

USER

동물 조회 · 입양 신청 · 봉사 · 후원

SHELTER_ADMIN

동물 등록 · 입양 승인 · 봉사 모집 관리

SUPER_ADMIN

전체 유저 · 게시판 · 입양 현황 통합 관리

트러블슈팅

/ JWT Claims 역직렬화 타입 불일치 문제 해결

BEFORE — 문제 코드

```
// userId를 Long으로 직접 캐스팅
Long userId = claims.get("userId", Long.class);

// 소형 숫자(1~9 등)는 Integer로 파싱됨
// → ClassCastException 발생
```

⚠ ClassCastException: Integer → Long 캐스팅 실패

AFTER — 수정 코드

```
// Number 인터페이스로 받아서 longValue() 변환
Object userIdObj = claims.get("userId");
Long userId = (userIdObj instanceof Number)
    ? ((Number) userIdObj).longValue()
    : null;
```

● 문제

특정 사용자 요청에서 `ClassCastException` 발생.
userId가 있는데도 인증이 실패하는 상황이 간헐적으로 재현됨.



● 원인

JJWT 라이브러리가 JSON 숫자를 역직렬화할 때
값 크기에 따라 `Integer` 또는 `Long` 으로 자동 결정.
작은 ID값(1~127)은 `Integer` 로 파싱 → `Long` 캐스팅 실패.

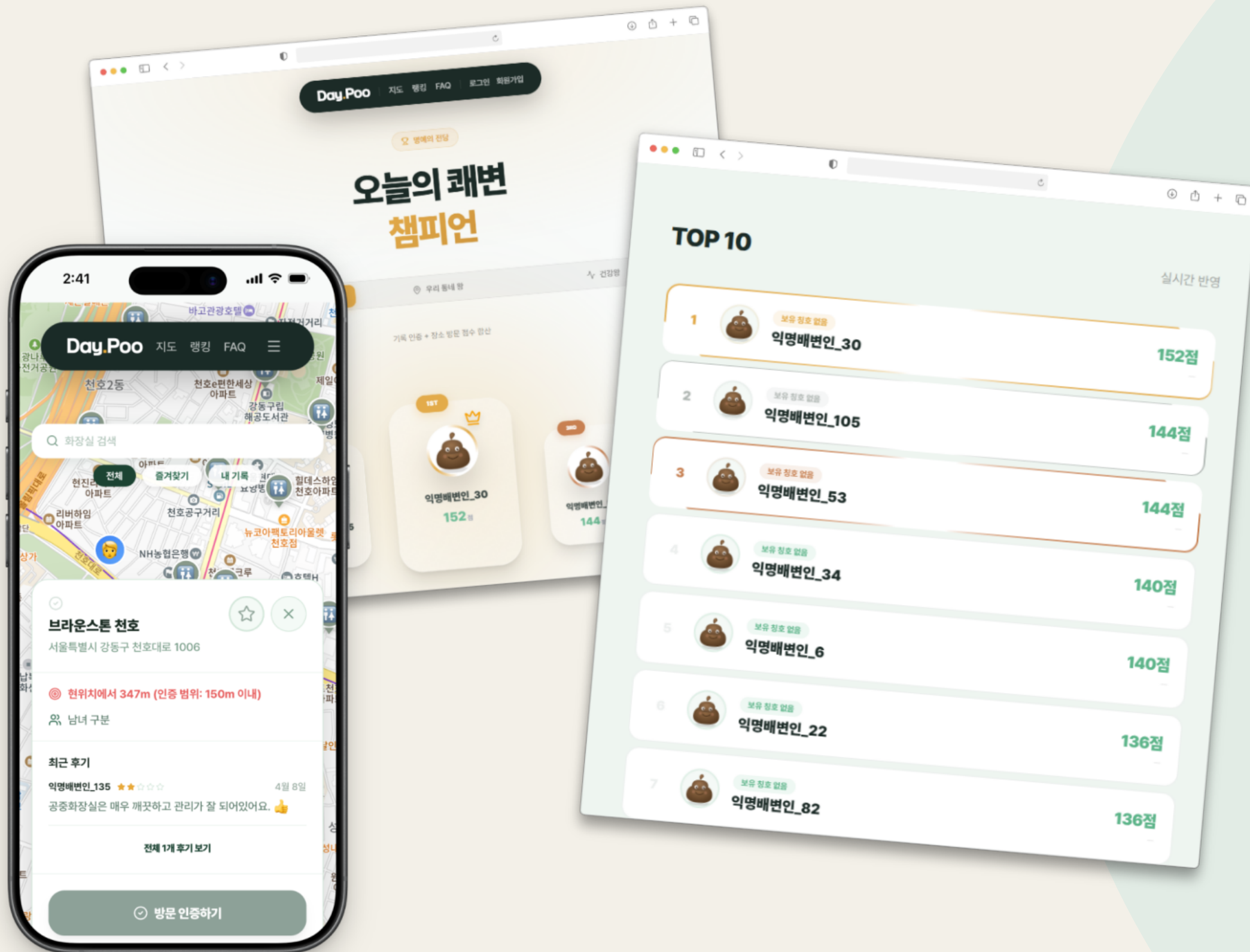


● 해결

`Long.class` 직접 캐스팅 대신 `Number` 인터페이스로 수신 후
`.longValue()` 로 변환 — 타입에 무관하게 안전하게 추출.
`null` 체크로 클레임 누락 상황도 방어 처리.

DAYPOO

실시간 공공데이터 기반 건강관리 서비스



기간

2026.03.19 ~ 2026.04.09 (3인 팀 프로젝트)

역할

프론트엔드 개발 / UI·UX 설계

핵심

번들 66% 감소 및 실시간 데이터 처리 구현

주요 기여

성능 최적화

Vite 기반 번들 66% 감소로 초기 로딩 속도 개선

실시간 데이터 처리

SSE 기반 인증 구조 설계 및 보안 문제 해결

인터랙션 구현

60fps 애니메이션 UI 구현 (Tailwind 활용)

데이터 처리

Redis 기반 실시간 랭킹 및 건강 리포트 구현

PWA 적용

iOS 대응 및 크로스 플랫폼 환경 구축

번들 최적화

Vite 전환으로 초기 로딩 속도 개선

BEFORE · CRA		AFTER · VITE	
번들 크기	~1.8 MB	번들 크기	~620 KB
초기 로딩	~3.2 s	초기 로딩	~1.1 s
빌드 시간	~45 s	빌드 시간	~8 s

번들 크기	1.8 MB → 620 KB	▼ 66%
초기 로딩	3.2 s → 1.1 s	▼ 66%
빌드 시간	45 s → 8 s	▼ 82%

적용 기법

- V

Vite 빌드 전환

CRA → Vite, esbuild 기반 네이티브 ESM으로 HMR 속도 향상

esbuild · Rollup
- {

트리 셰이킹

lodash → lodash-es 전환으로 미사용 코드 빌드 시 자동 제거

ES Module
- </>

코드 스플리팅

React.lazy + Suspense로 페이지 단위 동적 임포트, 초기 청크 최소화

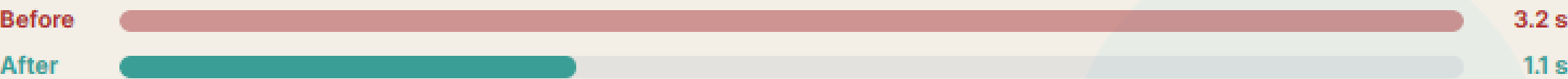
Dynamic Import
- []

청크 캐싱 전략

vendor / map 라이브러리 청크 분리, 재방문 시 캐시 히트율 향상

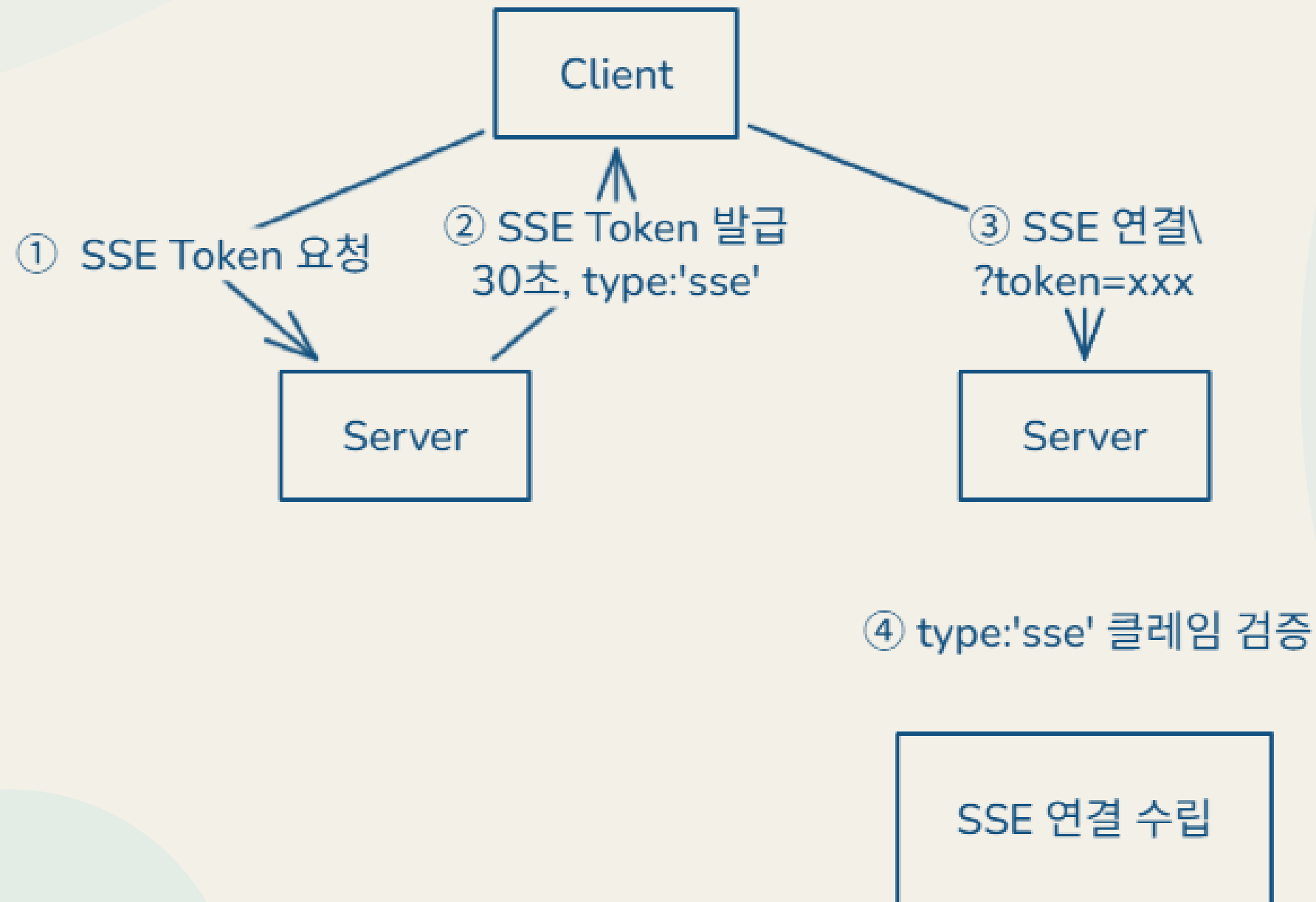
manualChunks

초기 로딩 비교



SSE 인증 문제 해결

EventSource 제약을 고려한 인증 구조 개선



- 문제

SSE 연결 시 Authorization 헤더 기반 인증 불가



- 원인

EventSource API는 브라우저 표준으로 커스텀 HTTP 헤더 설정 불가능



- 해결

- SSE 전용 단기 토큰(30초) 발급
- JWT에 type:'sse' 클레임 추가
- 일반 API 접근 차단

Thank you

감사합니다.

T. 010-8722-9185
uou9677@gmail.com